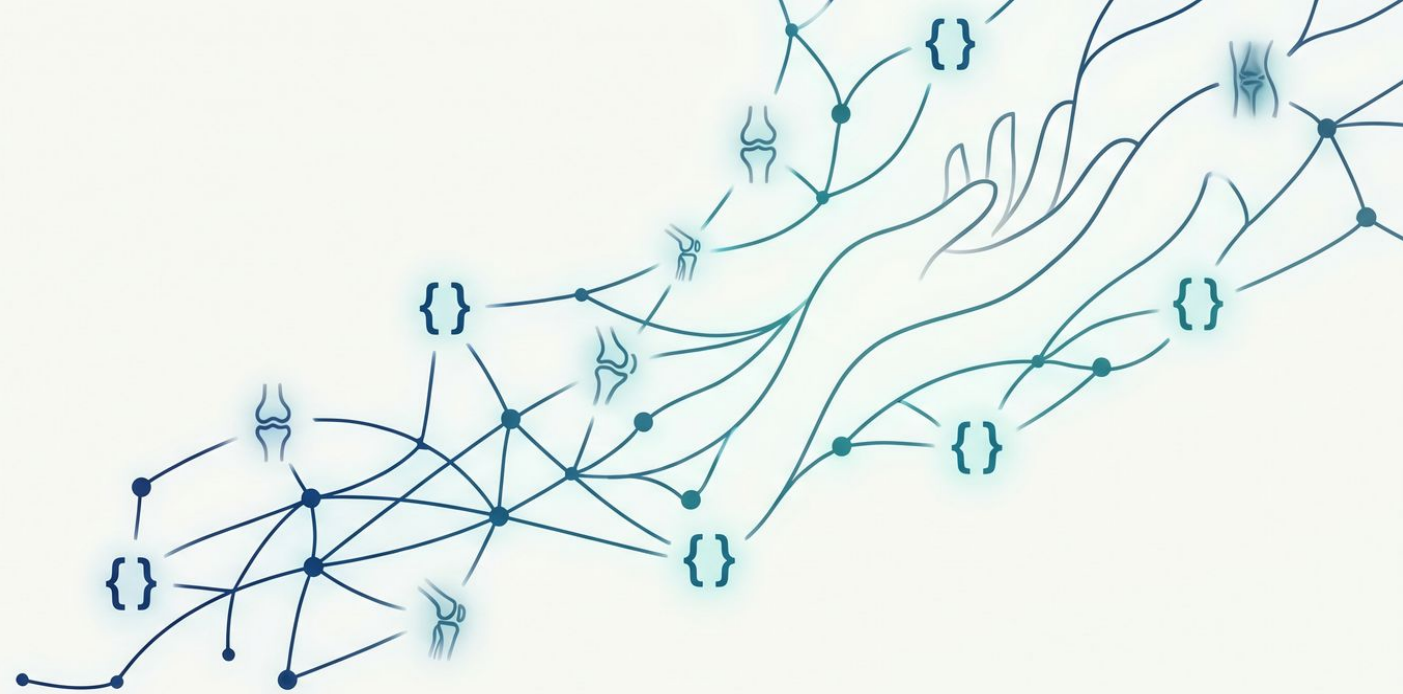




Best practices in maintaining and contributing to open source packages

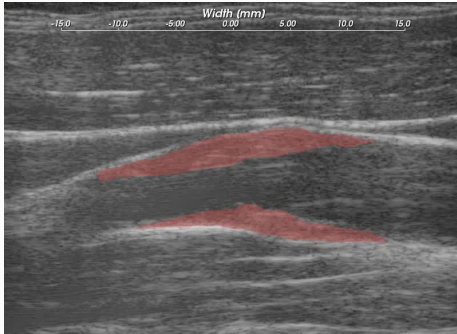
Lessons learned from real life



Matt McCormick, PhD
Founder, Fideus Labs

What is my story?

1

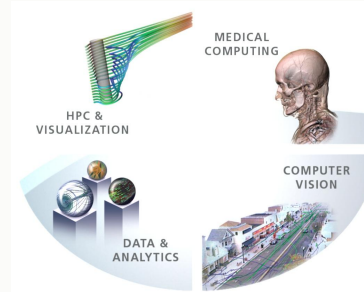


Biomedical Engineering

PhD, UW–Madison

Characterizing carotid artery biomechanics with diagnostic ultrasound. Signal processing, imaging physics, and neuroscience.

2

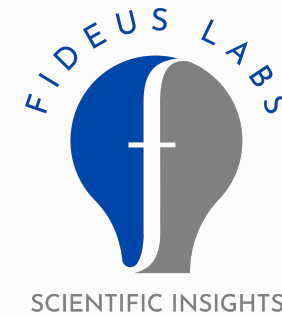


Distinguished Research Software Engineer

Kitware, Inc.

Computational bioimage analysis and materials science analysis. Curating the ITK community since 2012.

3



Founder

Fideus Labs LLC

Mission: build trust and advance understanding from bioimages.

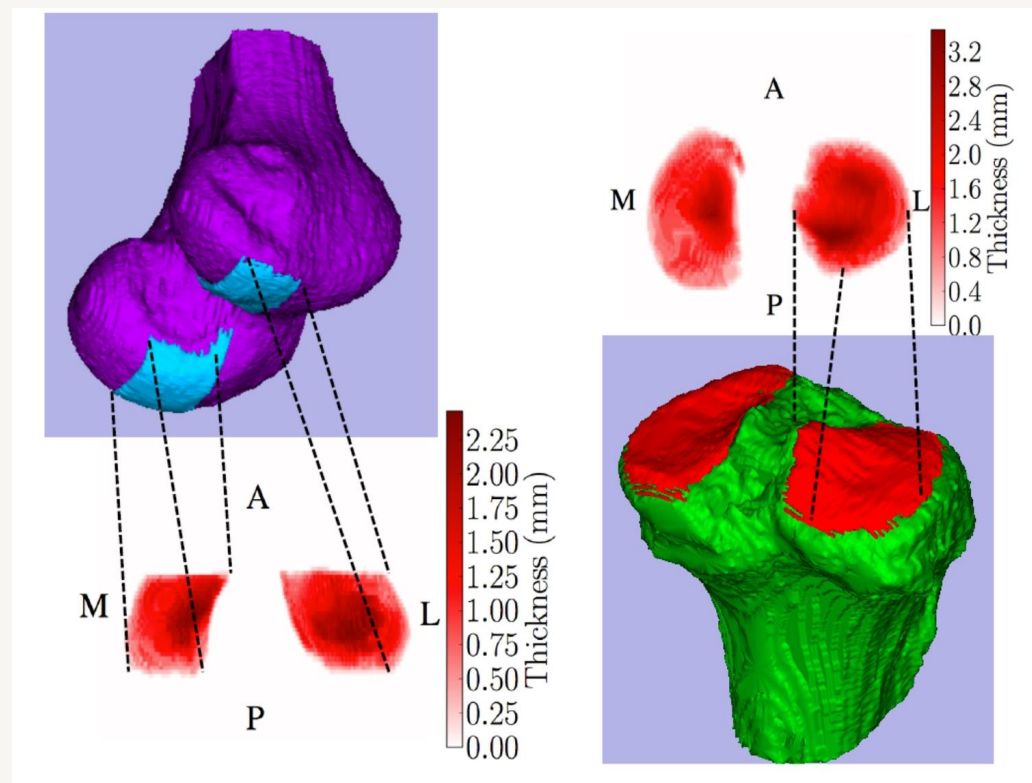
What have I kept coming back to?



The Insight Toolkit (ITK)

1999	Born to support analysis of the NIH Visible Human Project.
Open source	A cross-platform C++ / Python library for N-dimensional scientific image analysis — registration, segmentation, and more.
Foundational	Basis for many U.S. commercial products and university medical-image-analysis courses worldwide.
Community	Sustained since 2012 by welcoming contributors, documentation, and shared standards.

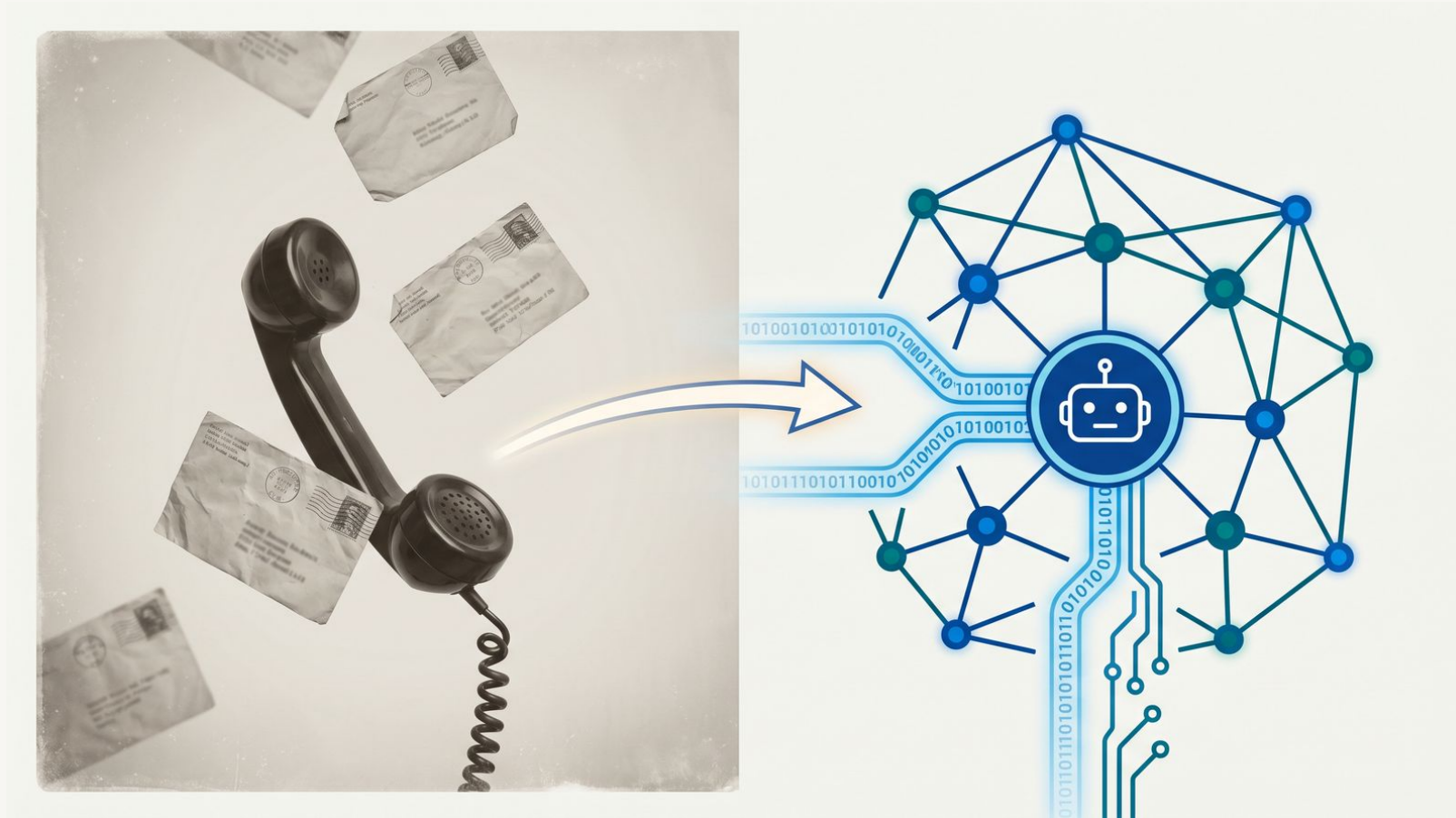
What does ITK actually do?



Automated knee-cartilage segmentation — the kind of quantitative MSK analysis ITK enables.

HOW WE WORK HAS CHANGED

From mailing lists to agents



When I started: literal teleconferences and GNU mailing lists. **Today:** agentic engineering on GitHub.

WHAT WILL WE COVER?

A few timeless best practices

For maintaining and contributing to scientific open source.

1

Contributing

How to be an effective contributor to research software.

2

Maintaining

How to keep a project healthy, open, and alive.

3

Agentic engineering

Hot takes on LLM coding agents — with data from ITK.

PART 1

Best practices for contributing

Become an effective contributor to research software — and grow your career while you do it.

Learn software carpentry

I learned by doing — with a community. You can too.

The Hacker Within

A grad-student peer-teaching group at UW–Madison, sharing scientific-computing best practices. This is where it clicked for me.

Software Carpentry

Foundational lessons — the shell, version control, testing, and reproducible workflows — for researchers who code.

Workshops like this one

ORMIR gathers the community to learn together. Yes — this is exactly how the skills spread.



Source: [The Hacker Within](#), [Software Carpentry](#)

Where the skills are passed on

Software Carpentry Boot Camp

Jan. 12 - 15, 2009



Day 1 - The Command Line - Navigating the Shell - Common Commands Day 2 - Shell Environment - Environment Variables - Basic Scripting Day 3 - Compiling / Linking - Libraries - Make Files - Using autoconf - Regular Expressions - grep, sed, awk Day 4 - Advanced Topics - Processes - Debugging - Common Utilities - Gnuplot - Python - LaTeX, Evince, EOG, GIMP	What: Four two-hour sessions serve as an introduction to basic UNIX functionality for the purpose of scientific software development. When: Jan 12-15, 2009 1:30-3:30 p.m. Where: CAE Lab 175 Who: This event is led by the Hacker Within, a scientific software development interest group at UW, and is intended for all levels of aspiring UNIX users and software carpenters. How: Register by sending an email to: hacker.within.admin@gmail.com And visit our google group at groups.google.com/group/hacker-within
--	--

Note! This event is contingent upon your participation and cannot take place without a sufficient minimum enrollment. So, please register early and share this invitation with others!

An early Software Carpentry bootcamp — researchers learning to code, together.

The Hacker Within Python Bootcamp 2010



A three day workshop introducing Python syntax, tools and techniques for scientific and research programming.

When: 10:30 - 4:30 pm
January 12, 13, and 14

Where: Mechanical Engineering Rm 1106

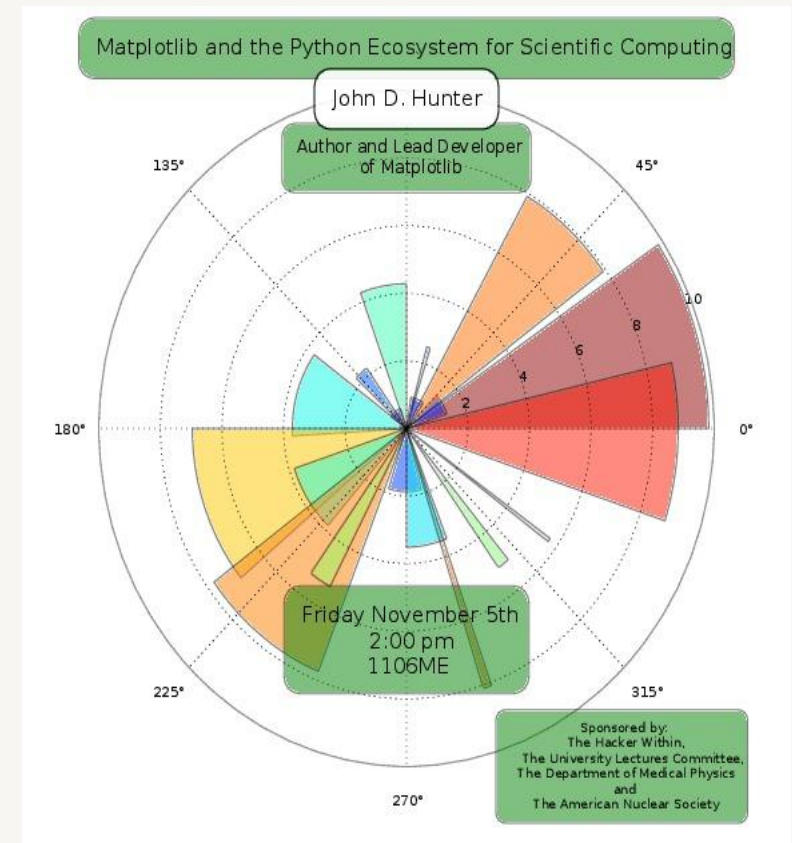
What: A bottom to top Python course covering basic syntax, data structures, functions, and libraries. Specific topics include the Scientific and Numeric Python libraries, the Matplotlib plotting library, statistics utilities, and web applications like the Google App Engine, Django, and Trac.

Who: Students comfortable with fundamental programming concepts such as variables, functions, loops (for, while), and conditionals (if, else).

How: Register online at: python.hackerwithin.org/register

For more information, visit the website: python.hackerwithin.org or email hacker.within.admin@gmail.com

Peer teaching in action — the room, not the lecture, is the point.



John D. Hunter, creator of Matplotlib — a Hacker Within talk that shaped scientific Python.

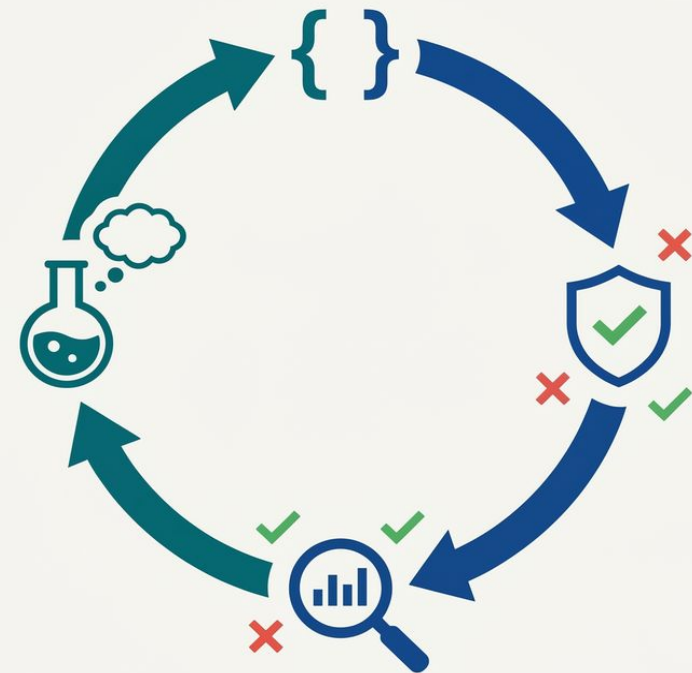
Practice continuous integration testing

Tests mirror the scientific method.

A test is a hypothesis about how your code should behave. Test-driven practice turns every claim into something reproducible and falsifiable.

CI catches regressions automatically.

Continuous checks run on every change, so the whole community can contribute with confidence that nothing quietly broke.



A test is a hypothesis. Continuous integration makes it reproducible.

Make your research software citable

Publish your data

Deposit datasets on Zenodo (an ORMIR community exists) or Figshare — each release earns a citable DOI.

Publish in JOSS

The Journal of Open Source Software gives your package a peer-reviewed, citable paper — developer-friendly and open access.

Use CITATION.cff

A machine-readable file in your repo tells the world exactly how to cite your work. GitHub renders it automatically.



In academia, citations are currency. Make your code count.

Source: [JOSS](#), [Zenodo](#), [CITATION.cff](#)

PART 2

Best practices for maintaining

Keeping a scientific project healthy, open, and alive — long after the first grant ends.

Give people agency

Strive for permissionless contribution.

Open a GitHub PR. Upload data. Publish a module — without waiting on a gatekeeper. Lower the barrier and people build.

The ITK Remote Module story.

Anyone can publish a Remote Module — new algorithms live in their own repository yet install seamlessly into ITK. Contributors keep ownership and credit.

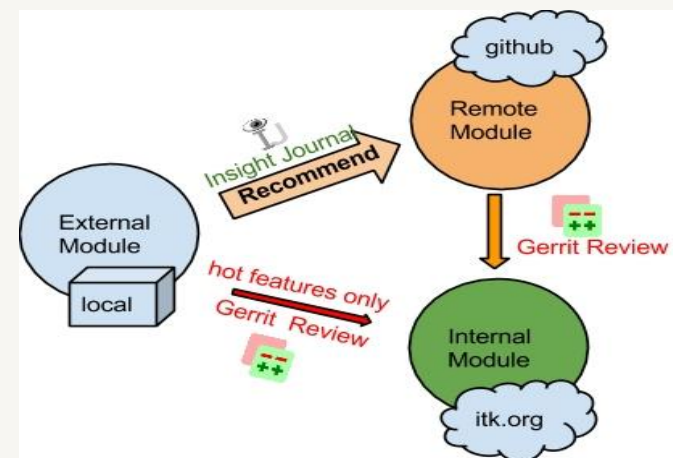
The best thing a maintainer can do is get out of the way.

Source: [ITK Remote Modules](#)

59

Remote Modules

shipped with ITK 5.4 — each one a permissionless community contribution.



Permissionless, by design

Own repository

The contributor's code stays in their repo — their license, their release cycle, their credit.

Seamless install

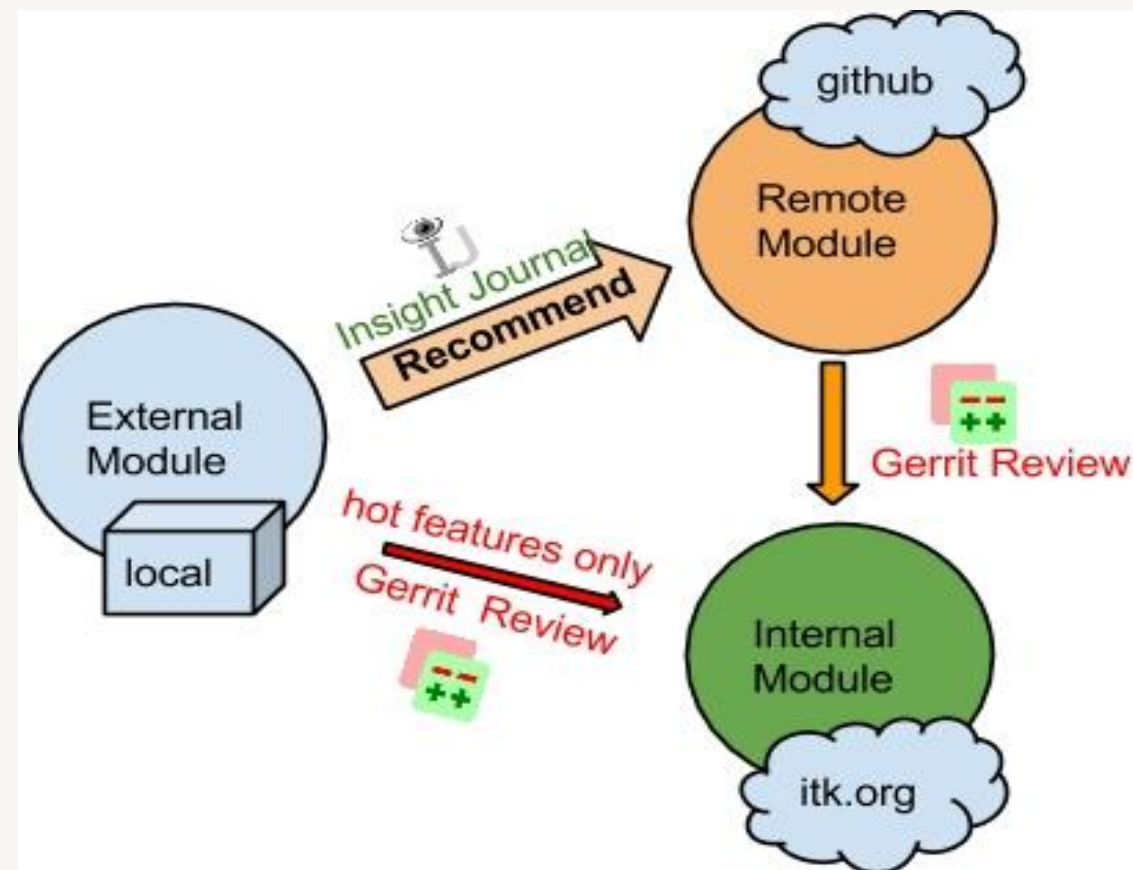
A one-line entry lets ITK users pull the module in at build time, as if it were core.

No gatekeeping

The maintainers don't have to review every algorithm into the core — innovation isn't bottlenecked on us.

Scales the community

The result: 59 modules, each an author who felt ownership rather than friction.



Source: [ITK Remote Modules](#)

What happens when the grant runs out?



Complexity is the enemy of longevity. Build lean.

Build on simple, low-cost infrastructure

Choose tools the project can sustain forever

Free CI

Use freely available CI testing — e.g. GitHub Actions — instead of servers you have to run and pay for.

Static hosting

Prefer static website hosting. It is cheap, fast, and effectively maintenance-free.

Plan for after the grant

Choose infrastructure the project can sustain once funding ends — so the work continues.

The Insight Journal story

An open, reproducible publication venue that outlived its original funding by keeping infrastructure lean.



Source: [GitHub Actions](#), [Insight Journal](#)



**Insight
Journal**

A Legacy of Innovation and Impact

Advancing Medical Image Analysis through Open Science



First Open-Access Journal in
Medical Image Analysis



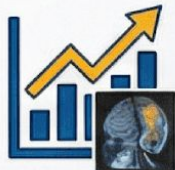
Complete Reproducibility:
Code, Data, & Reviews included



**Transparent Open Peer
Review System**



Community-Driven by
the ITK Community



Hundreds of articles
advancing the field



Technical Debt Led to System Failure

The Sustainability Crisis of the Original LAMP Stack

Critical Challenges

- **Maintainability Crisis:** Legacy PHP codebase difficult to update
- **Infrastructure Costs:** High maintenance for servers & database
- **Security Risks:** Aging stack vulnerable to exploits
- **Scalability Limits:** Incompatible with modern web standards

Consequences

- Website became non-functional
- Submissions and reviews stopped
- Valuable research at risk of loss



DeSci Principles Embedded in IJ3



- **Distributed Storage:** IPFS ensures content survival; "many copies keep things safe"



- **Cryptographic Provenance:** Content-addressed identifiers (CIDs) for tamper-proof verification.



- **Community Ownership:** No corporate publishers, paywalls, or gatekeepers.



- **Permanent Accessibility:** Research remains available forever, independent of institutions.



Advancing Science Through Decentralization and Openness.

Acknowledge people — often and generously



Open source runs on recognition. Give it generously.

Physical mementos

ITK sends contributors a small 3D-printed token for each release — a tangible thank-you people keep on their desk.

Release notes

Call out contributors by name in every release. Recognition is the currency of open source.

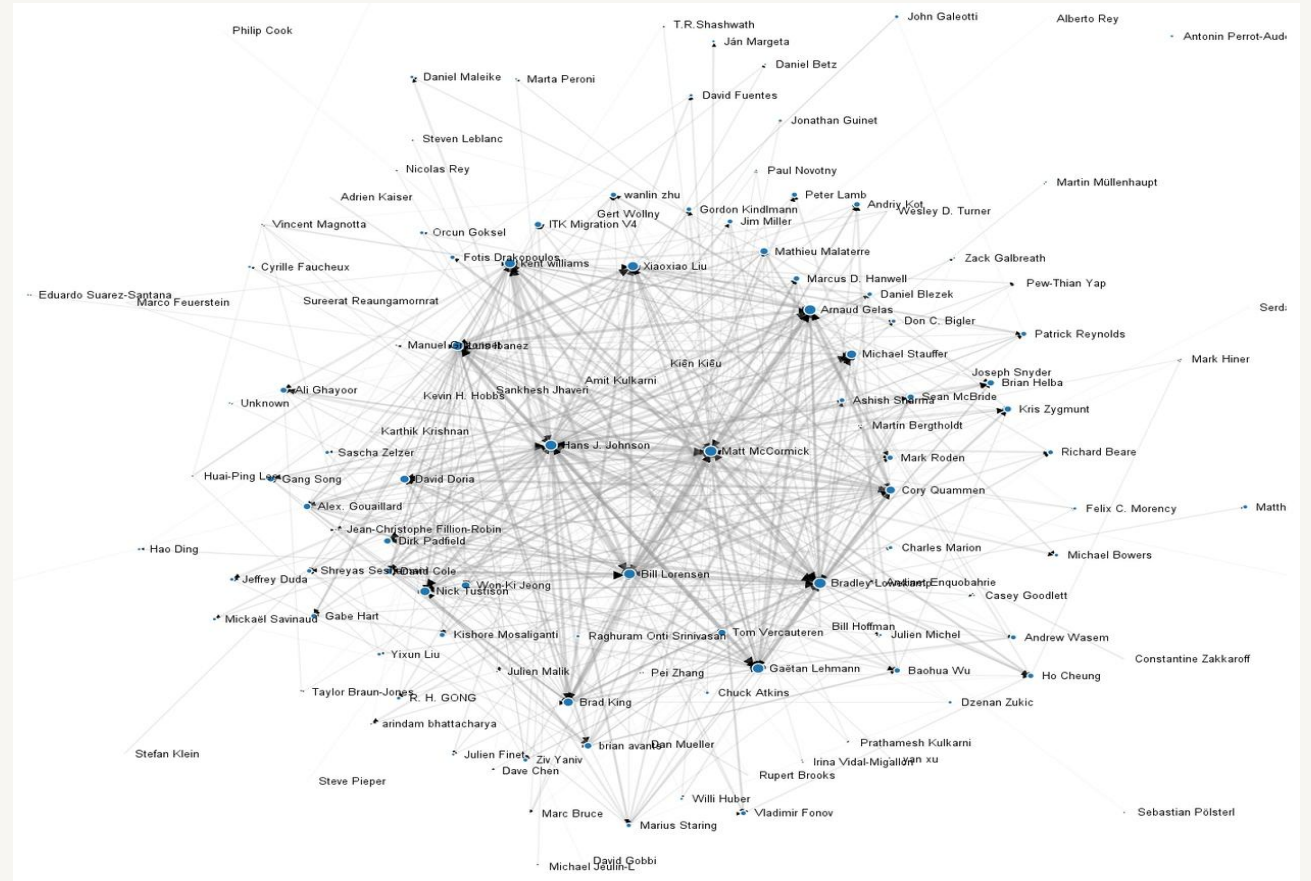
Recognize by ORCID

A .zenodo.json file credits contributors by ORCID, so their work is formally attributed on every archived release.

Recognition, made tangible



The ITK contributor memento — a physical thank-you for contributions a release.



Every name is a real person. A collaboration graph of ITK contributors — a community, not a codebase.

PART 3

Hot takes on agentic engineering

A new discipline is reshaping how we build scientific software —
and ITK has the receipts.

What is agentic engineering?



Not just typing prompts — a genuine engineering discipline.

What is agentic engineering?

Effective software engineering with LLM coding agents.

Context engineering

Give the agent the right information — architecture, conventions, glossary — so it makes the right decisions.

Test-driven development

Let tests define correctness.
Agents iterate fast; tests keep them honest.

Effective review agents

Use agents to review agents.
Automated code review scales human judgment.

Does it actually work? Let's look at ITK.

On November 6, 2025, ITK added its first **AGENTS.md** — released two weeks later as **ITK 5.4.5: “Onboarding AI Agents.”** I compared the ~8 months since against the entire decade before.

2.3×

issues resolved / month

2.7×

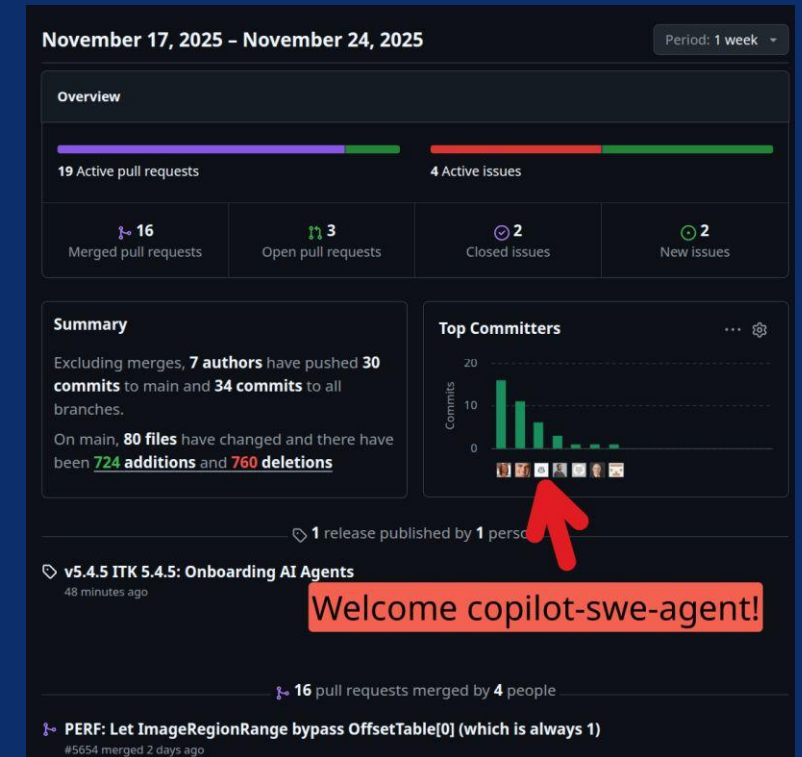
PRs merged / month

2.5×

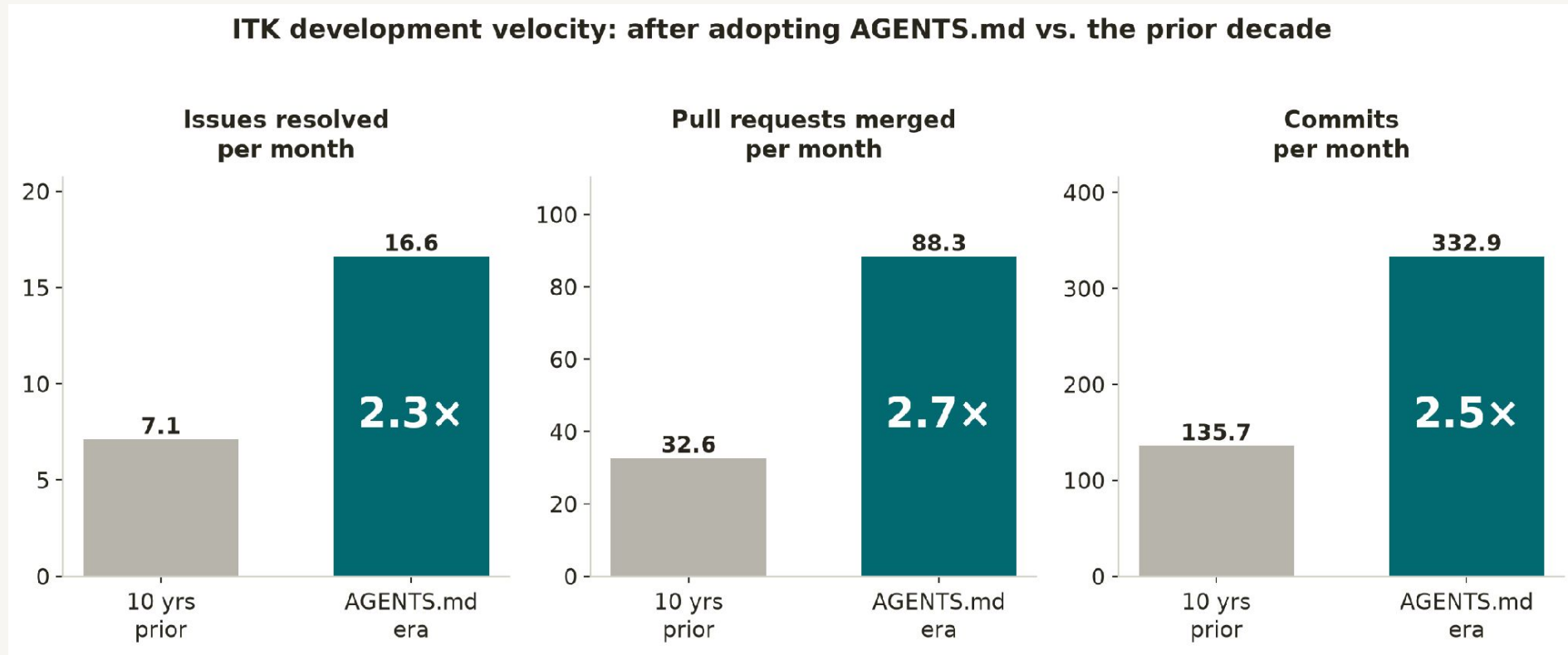
commits / month

Normalized monthly rates: AGENTS.md era (Nov 2025–Jul 2026) vs. the prior 10 years.

Source: [ITK GitHub](#)



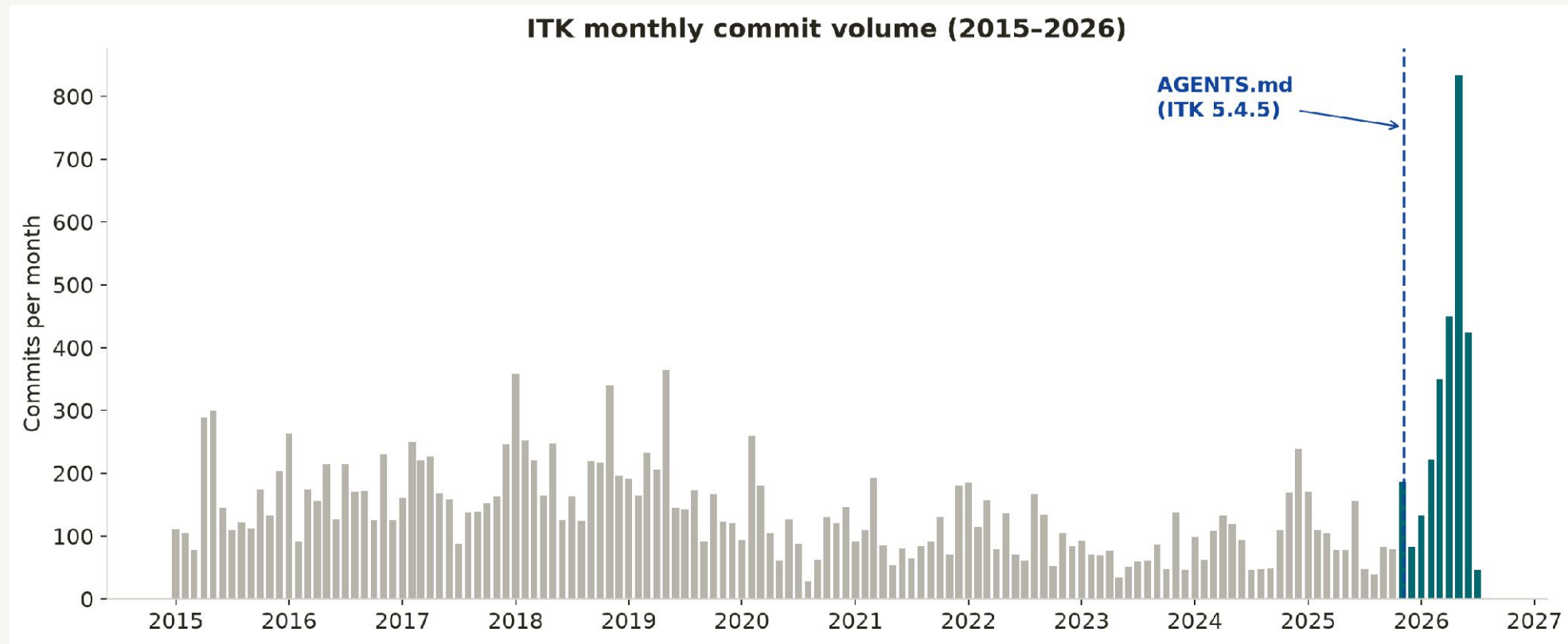
Velocity: after AGENTS.md vs. the prior decade



Per-month rates normalize the very different window lengths (~8 months vs. 120 months) for a fair comparison.

Source: [ITK GitHub — commits & Search API](#)

Ten years of commits — then a step change



Monthly commit volume held roughly flat for a decade, then climbed sharply after AGENTS.md — peaking above 800 commits in a single month.

Source: [ITK GitHub](#)

But hold on — a few honest caveats

Correlation ≠ causation

The team also grew more agent-fluent over this window. AGENTS.md is a marker, not the sole cause.

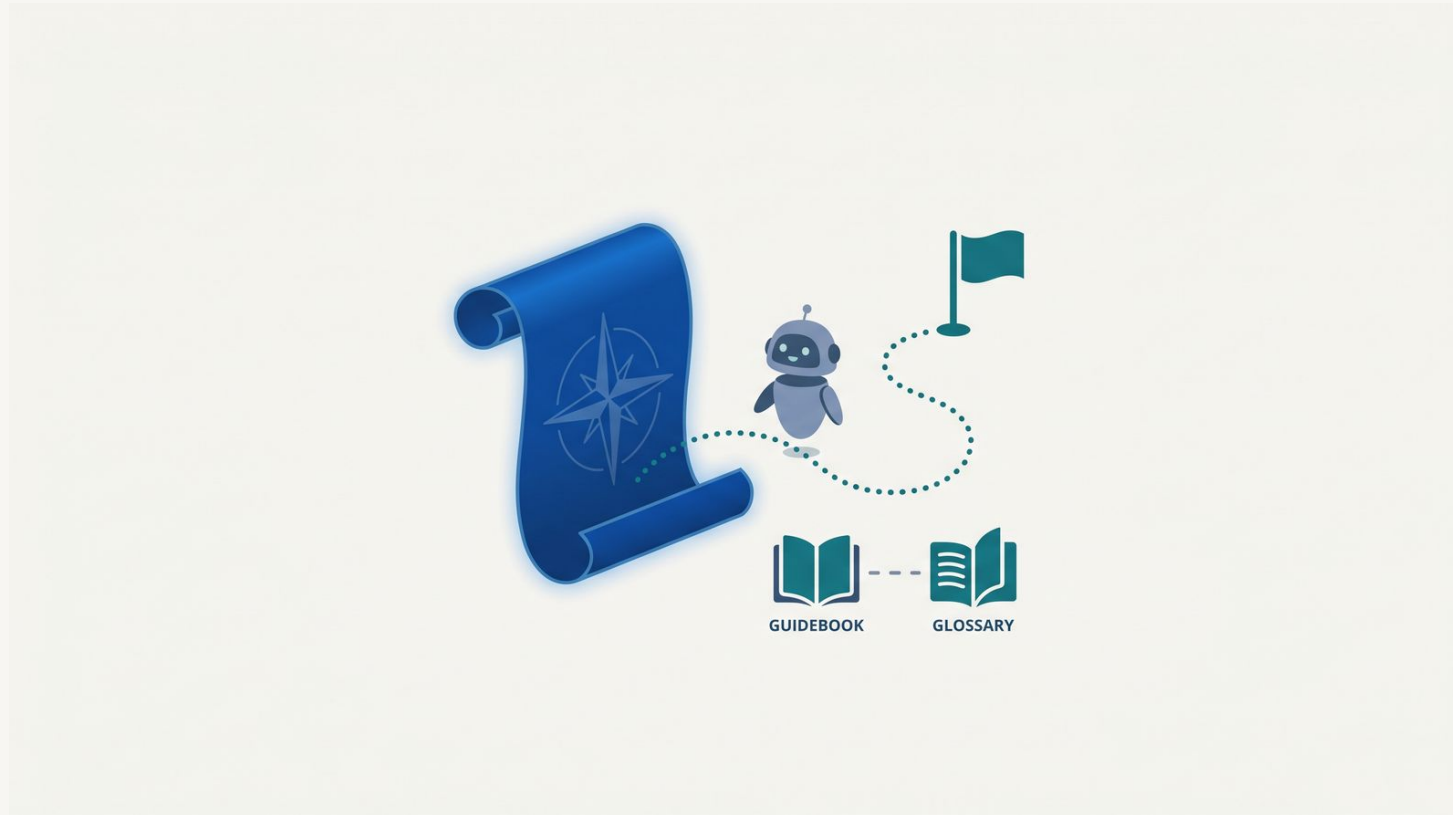
Volume isn't value

More commits and PRs don't automatically mean better science. Quality still needs human judgment.

Guardrails did the heavy lifting

This only worked because ITK already had strong tests and CI. Agents amplify good practices — and bad ones.

Why does one Markdown file matter?



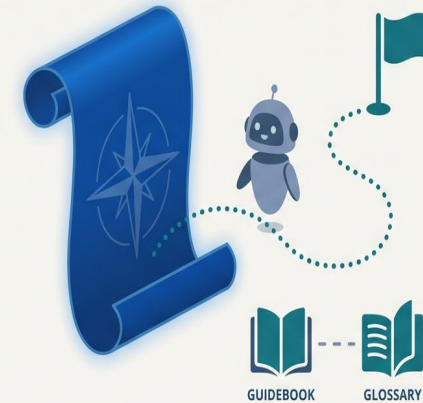
AGENTS.md is the README for AI — and for humans, too.

Context engineering is the game

AGENTS.md is the map.

An open, vendor-neutral file that tells any coding agent how a project is built, tested, and structured — the README for AI.

It captures development practices *and* project understanding — so agents (and newcomers) start from shared ground.



Source: [AGENTS.md](#)

Could AGENTS.md be ORMIR's superpower?

A shared context layer is a natural fit for a standards community.

A common glossary.md

Consistent ORMIR terminology across every project — so humans and agents speak the same language.

Notebook guidelines

Sourced from AGENTS.md: how to structure reproducible notebooks the ORMIR way.

Community data standards

ORMIR-MIDS conventions, file formats, and metadata — encoded once, reused everywhere.

A single, versioned source of truth can be a critical role for ORMIR — for people and agents alike.

Source: [ORMIR-MIDS](#), [ORMIR](#)

What should you take home?

Contributing

- Learn software carpentry with a community
- Practice continuous integration testing
- Make your work citable: Zenodo, JOSS, CITATION.cff

Maintaining

- Give people agency go permissionless
- Build on simple, low-cost, free infrastructure
- Acknowledge contributors, often and generously

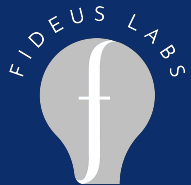
Agentic engineering

- Context engineering + TDD + review agents
- ITK: ~2.5× more activity after AGENTS.md
- AGENTS.md could be a superpower for ORMIR

IN CLOSING

We make an impact through open science.

And we make open science happen through **high-quality research software** and **community**.



Matt McCormick, PhD

Fideus Labs

<https://fideus.io>

